

Cowboy Secrets: CBSS

Technical Whitepaper

Charles DePue, Cowboy Foundation

May 2026 — Draft v0.1.0

1 Abstract

This document specifies the **Cowboy Secret Service (CBSS)**: the chain-native subsystem that lets account owners store API keys, model credentials, webhook tokens, and other application secrets on Cowboy and have them released, on demand, into the address space of a runner that has been dispatched to execute a specific job for a specific actor — without the chain, the runner host, the dispatcher, or any individual proxy ever holding the plaintext on its own.

CBSS is a **threshold identity-based encryption (IBE)** scheme over BLS12-381, operated by a permissioned set of staked **CBSS Proxies** (the `cbssd` daemon). Each account has a per-account **release key** generated by a Distributed Key Generation (DKG) ceremony. To wrap a secret, the owner's CLI computes an identity from the secret's (`account`, `key_hash`, `version`, `wrap_epoch`) tuple, derives a one-time AES key from $e(I, MPK)$, and AES-256-GCM-encrypts a fresh Data Encryption Key (DEK) into a chain-anchored **WrappedDek**. The plaintext payload itself is encrypted under the DEK and stored as a CBFS object (CIP-9). To unwrap, a runner constructs a signed release request, fans out partial-sign RPCs to t of n committee proxies, Lagrange-combines the partial BLS signatures into the threshold signature $\sigma = MSK \cdot I$, and recovers the same AES key via $e(\sigma, G2)$ to decrypt the DEK and (separately) the CBFS payload.

Together with CBFS (CIP-9) and the runner marketplace (CIP-2), CBSS gives Cowboy three properties that prior decentralized-secrets systems each provide one of: (1) **on-chain authorization** for every release (chain-signed receipts pin the responsible runner, actor, job, and policy snapshot); (2) **off-chain plaintext** (no validator, dispatcher, or single proxy ever sees the DEK or payload); (3) **runner-scoped delivery** (secrets land in the address space of one runner for one job, never in process environment or shared cache). The data path involves no on-chain transaction; the chain is touched only at lifecycle boundaries (set, finalize, policy update, delete, register, rotate, slash, receipt). This specification is a complement to the [Cowboy Technical Whitepaper](#) and the [Cowboy Storage Whitepaper](#); where this document and the Technical Whitepaper conflict, the Technical Whitepaper prevails.

2 Introduction

Autonomous agents on Cowboy routinely call third-party APIs — model providers, search APIs, payment rails, observability backends, MCP tool servers — that authenticate by bearer token. Today the only mechanism a Cowboy actor has for getting that bearer token in front of a runner is to put it in calldata, which puts it in front of every validator and every block-explorer scraper for the rest of consensus history. Equivalently: an actor can put the secret in clear in CBFS and

trust a single runner not to leak it. Both options collapse to “you trusted somebody,” and the *somebody* is the wrong shape for an autonomous, self-funding, self-scheduling program.

The fundamental problem is that smart-contract platforms have no native notion of conditional plaintext release. Ethereum’s storage is public; even private rollups make every encrypted byte equally available to whoever decrypts the rollup. Validators must agree on state; if validators must agree on plaintext secrets, plaintext secrets are not secret. Conversely, off-chain secret stores (Hashicorp Vault, AWS Secrets Manager, doppler.com) do release plaintext conditionally, but their authorization model is “an HTTPS bearer token from a long-lived service account,” which has no link to the chain’s notion of who is dispatched to do what.

CBSS sits between these two regimes. The **authorization** lives on chain: an actor’s manifest declares which secret keys it intends to read; the chain mints a job, pins a runner, and emits a release request signed by the runner. The **plaintext path** lives off chain: the runner fans out partial signatures from a threshold committee, combines them locally, and decrypts the DEK and payload entirely in its own process. The **economic enforcement** also lives on chain: each participating proxy submits a chain-verifiable receipt that pairs the partial signature against the proxy’s published Verifiable Secret Sharing (VSS) commitment. The chain therefore has, after the fact, a complete signed audit trail of who released what to whom for which job, but at no point during the release does the chain or any single party hold the secret.

This document specifies the cryptographic construction, the on-chain instruction set (CIP-24 opcodes 68–81), the `cbssd` proxy daemon, the runner-side `secrets-manager` client, the owner-facing CLI flows, the gas pricing, the slashing model, and the parameter table. It is the normative reference for implementations of CIP-24.

3 Architectural Overview

This section is descriptive and non-binding. Normative requirements are in §§1–13.

3.1 Terminology

- **Secret:** A named, account-owned, versioned byte string (e.g. `OPENAI_API_KEY`). The chain never sees the plaintext.
- **Owner:** The Cowboy account that owns one or more secrets. The owner’s wallet key authorizes `SetSecret`, `UpdateSecretPolicy`, and `Delete*` transactions.
- **Actor / Runner / Job:** As in the Technical Whitepaper. An actor declares secrets it intends to read in its manifest; a job is dispatched to a specific runner; that runner is the one entitled to pull the declared secrets for the duration of the job.

- **CBSS Proxy:** A staked off-chain participant running the `cbssd` daemon. Proxies hold one BLS scalar share per release-key scope they serve and produce partial threshold signatures on demand.
- **Committee:** The ordered set of proxies serving a particular release key. Default 3-of-5; configurable up to 16 with `--committee-override`.
- **DKG (Distributed Key Generation):** The multi-round protocol that produces the committee's Master Public Key (MPK), per-proxy shares, and per-proxy VSS commitments without any single proxy ever holding the Master Secret Key (MSK).
- **PSS Reshare:** Proactive Secret Sharing. A protocol that replaces every committee share with a fresh share for the same MSK, optionally over a different committee, without ever materialising the MSK.
- **Identity (I):** A point in G_1 derived deterministically from `(account, key_hash, version, wrap_epoch)`. Each secret version under each `committee_epoch` has a unique identity.
- **MPK / MSK:** Master Public Key ($MPK \in G_2 = MSK \cdot G2_gen$) and Master Secret Key ($MSK \in \text{Scalar}$, never instantiated). Preserved across reshares.
- **Wrapped DEK:** AES-256-GCM ciphertext of a Data Encryption Key, encrypted under the symmetric key derived from the IBE pairing $e(I, MPK)$.
- **CBFS pointer:** A CIP-9 object descriptor pointing to the AES-256-GCM ciphertext of the actual secret payload, encrypted under the DEK.
- **Release request:** A runner-signed structure naming the secret, version, actor, runner, job, release nonce, request block, and serve epoch. Hashed and signed under domain separator `cbss/release-request/v1`.
- **Partial signature (σ_i):** $s_i \cdot I$ where s_i is a proxy's share. Verified individually via the pairing $e(\sigma_i, G2_gen) == e(I, S_i)$ against the on-chain VSS commitment S_i .
- **Threshold signature (σ):** Lagrange combination of t partial signatures, equal to $MSK \cdot I$.
- **Receipt:** A chain-submitted `(release_request, sigma_i, proxy_id, committee_epoch_at_serve, served_at_block, proxy_sig)` that earns the proxy a per-receipt fee and constitutes the canonical audit record.
- **CBSS System Actor (0x04):** The Cowboy system actor that owns secret metadata, release keys, proxy registry, DKG-pending records, release receipts, liveness challenges, and slashing.

3.2 Key Features

CBSS provides six core technical features:

- **Off-chain plaintext data path.** Wrap is local to the owner; unwrap is local to the runner. The chain stores ciphertext (`WrappedDek` and `cbfs_pointer`); plaintext exists only in the owner’s CLI memory at write time and the runner’s process memory at read time, both bounded by Rust `Zeroizing` wrappers.
- **Threshold IBE over BLS12-381.** The same primitive used by drand for time-locked encryption: identity-based encryption where the “identity” is a per-version tuple, encryption uses only the public MPK, and decryption requires t of n proxies to produce partial BLS signatures on the identity. No single proxy holds enough information to decrypt; collusion requires $\geq t$ proxies.
- **Per-account release keys with optional per-secret override.** The default path generates one DKG per account and reuses its MPK for every secret the account wraps. High-value secrets can opt into a separate DKG with custom committee selection (`--committee-override --threshold T/N`), trading extra latency at write time for committee diversity.
- **Chain-anchored authorization.** Every release is preceded by a runner-signed `SignedReleaseRequest` and followed by chain-recorded `ReleaseReceipts`. Policy (actor allowlist, TEE requirement) is checked at receipt-submission current head, so revocation propagates within finality plus proxy cache TTL.
- **Proactive secret sharing.** Reshares are scheduled (every `RESHARE_INTERVAL_BLOCKS`) and triggered by churn; PSS preserves MPK across reshares so historic ciphertexts remain decryptable, but every share is replaced — a previously-compromised share is invalidated by the next reshare.
- **Independent per-proxy receipt submission.** No coordinator runs the receipt phase. Each of t participating proxies submits a per-proxy receipt independently, with up to 10 blocks of jitter to smooth chain throughput. The chain emits one `ReleaseReceiptRecorded` event per receipt and exactly one aggregate `SecretReleased` event when the t -th lands. Runners do not gate or sign-off the audit.

3.3 Differences vs. Existing Secret Stores

Aspect	Hashicorp Vault / AWS Secrets Manager	NuCypher / Lit Protocol	CBSS
Authorization root of trust	Service-account token	TaCo-style ABE / external policy	Cowboy chain (CIP-2 job dispatch)

Aspect	Hashicorp Vault / AWS Secrets Manager	NuCypher / Lit Protocol	CBSS
Plaintext custodian	The vault server	Threshold KFrag holders	Owner at write, runner at read
Release latency	Single HTTPS RT	Multi-RT to KFrag holders	t parallel QUIC RTs (~100 ms)
Audit trail	Server logs	Off-chain	Chain receipts (per-proxy + aggregate)
Revocation	Token revoke; immediate	Re-encryption on policy change	Metadata-only; $\leq$ proxy cache TTL
Re-wrap on policy change	None	Yes (by design)	Never (per-account release key + ACL at read)
Committee rotation	N/A	Manual	Block-scheduled PSS reshare
Proxy slashing	None	None	On-chain SlashCbssProxy evidence
Native to actor model	No	No	Yes — secrets=[KEY] kwarg on runner ops

Three properties are jointly load-bearing for Cowboy and jointly absent from every alternative above:

1. **No re-encryption on ACL change.** Vault and Lit both require re-wrapping when the access list changes. CBSS uses a fixed per-account release key and validates `policy.actors` *at the moment a receipt is submitted*. Adding or removing an actor is a metadata-only `UpdateSecretPolicy` instruction; no DEK is re-wrapped, no payload re-uploaded.
2. **Scope-locked release.** A release fires for one (actor, runner, job, secret, version) tuple, with a release nonce. The chain receipt encodes that scope; off-chain leakage is therefore tagged to specific responsible parties rather than dissolving into “someone with a token.”
3. **Data plane has zero per-access chain transactions.** The owner CLI submits one transaction at write time; the runner submits zero at read time; only the participating

proxies submit (one each, asynchronously with jitter). For comparison, every Vault read is a service request to a centralized custodian; every Lit read is several HTTPS RTs to KFragment holders.

4 Related Work

4.1 Centralised Secret Stores (Vault, AWS Secrets Manager, Doppler, 1Password Secrets Automation)

Hashicorp Vault is the canonical model: a single service that stores ciphertext, authenticates clients via short-lived tokens, and releases plaintext over HTTPS. AWS Secrets Manager and Doppler are operationally equivalent. They are excellent at what they do — bearer-token authorization, zero-config rotation, fine-grained policy — and absolutely unsuitable for a public blockchain’s autonomous-agent layer:

- **Single point of compromise.** A vault breach exposes every secret it holds.
- **No on-chain authorization link.** A Vault token says “this account has read access”; it has no way to say “this runner address is dispatched to this job at this block height.”
- **No native audit on the consumer chain.** Vault audit logs live in the vault.
- **Operationally Web2.** A Cowboy actor that pulled secrets from Vault would need to bridge a service-account token into chain identity, defeating the point.

4.2 Threshold Cryptography for Decentralised Secrets (Lit Protocol, NuCypher / TACo, drand timelock)

NuCypher (now Threshold Network’s TACo) and Lit Protocol both use threshold cryptography to release secrets contingent on policy. They are conceptually closest to CBSS:

- **Lit Protocol** uses a permissioned BLS threshold network. A user encrypts under the network’s MPK with an attached “access control condition” (an EVM `eth_call`-style predicate); to decrypt, a client requests partial decryptions from t of n Lit nodes, which each evaluate the condition before contributing. The trust model is similar to CBSS but the on-chain anchoring is weaker — the access condition is evaluated by each Lit node independently against any chain it can read, with no on-chain receipt that the release happened.
- **NuCypher / TACo** uses proxy re-encryption (ECIES-based KFrags). Granting access requires generating new re-encryption keys and distributing them; revoking access requires re-encryption. The model is elegant for sharing-style workloads (Alice grants Bob access to

a document) but expensive for autonomous-agent workloads where an actor’s manifest may decide moment-to-moment which secrets it needs and the access list changes frequently.

- **drand timelock** uses tlock — threshold IBE keyed on future round numbers — to encrypt to a future time. CBSS reuses the same construction (BLS12-381 hash-to-curve identity + pairing-based KEM + AEAD) but keys identities on per-version-tuple rather than future rounds, and adds chain-anchored authorization, audit, and slashing.

CBSS adopts the threshold-IBE part of drand’s tlock and the chain-policy part of Lit’s design, but anchors authorization to Cowboy’s job-dispatch + runner-stake model rather than to an arbitrary `eth_call` predicate. Receipts and slashing are first-class.

4.3 Multi-Party Computation Secret Sharing (TSS Vaults, Pyroflex, ZenGo)

Threshold Signature Schemes (TSS) used by custody products like Fireblocks, ZenGo, and Coinbase MPC are mechanically related: they share a private key across nodes and produce signatures via threshold protocols. They are designed for *signing*, not for *encryption-decryption-of-arbitrary-payloads*. Re-purposing a TSS vault for secret release would require either re-encrypting payloads under a derived symmetric key the TSS produces (essentially CBSS’s construction in different clothing) or signing each payload-release request individually (functionally equivalent but with worse parallelism, since signing requires interactive rounds where IBE decryption can pre-commit identities).

CBSS uses BLS-IBE rather than TSS-ECDSA because the algebra is simpler (single pairing equation per partial verification, non-interactive after DKG), the verification is one pairing per partial (vs. multi-round MPC for ECDSA), and reshare is a clean PSS protocol on Shamir polynomials rather than a custom TSS resharing dance.

4.4 Hardware-Rooted Approaches (TEE-Sealed Vaults, Intel SGX SealedData)

Sealing secrets in a TEE (Intel SGX, AMD SEV-SNP, AWS Nitro) gives strong custody guarantees: the secret is unsealed only inside attested hardware. Cowboy already has a CIP-23 TEE attestation primitive; combining it with a centralized vault is straightforward. The reasons CBSS does not stop there:

- **TEE attacks.** Repeated side-channel breakthroughs against SGX (Foreshock, *ÆPIC*, Downfall, etc.) make any “TEE-only” custody model fragile. Threshold cryptography is a structural defense — compromising any $t-1$ proxies still discloses zero plaintext.

- **No vendor lock-in.** A multi-vendor proxy fleet running pure software is a deliberate property; threshold IBE is hardware-agnostic.
- **TEE composes additively.** A secret may carry `policy.tee_required = true` to require the *runner* to be TEE-attested at release time; the proxies themselves are not required to run in TEEs. Use TEE where it adds; don't depend on it.

4.5 Tabular Summary

System	Custody model	Per-release chain action	Audit on chain	Re-wrap on ACL change	Native to L1 actor model
CBSS	Threshold IBE, t-of-n staked proxies	0 (proxies submit asynchronously)	Per-proxy + aggregate events	Never	Yes
Hashicorp Vault	Centralised	N/A (off-chain)	No	No	No
AWS Secrets Manager	Centralised	N/A (off-chain)	No (CloudTrail only)	No	No
Lit Protocol	Threshold BLS (PoA committee)	None enforced	Off-chain	No	No
NuCypher / TACo	Proxy re-encryption (KFrag holders)	KFrag generation per grant	Optional	Yes (re-encryption)	No
drand timelock	Threshold IBE (round identities)	None	None	N/A	No
Fireblocks / ZenGo MPC	TSS (key sharded for signing)	None	Off-chain	N/A (signing primitive)	No
TEE-sealed vault	Single TEE	Attestation per release	Optional	Optional	No

5 Cryptographic Construction

CBSS implements the threshold IBE construction from drand’s `tlock` paper, specialised to Cowboy’s per-secret-version identities and bound to chain-anchored authorization. This section restates the math in terms of CBSS objects; cryptographers should treat it as a presentation of standard threshold-BLS / Boneh-Franklin IBE rather than a novel scheme.

5.1 Curve and Primitives

The curve is **BLS12-381**. We use:

- **G1**: 48-byte compressed points, used for identities `I` and partial / aggregate signatures σ_i , σ .
- **G2**: 96-byte compressed points, used for the generator `G2_gen`, the master public key `MPK`, and per-proxy VSS commitments `S_i`.
- **Gt**: target group of the pairing $e: G1 \times G2 \rightarrow Gt$.
- **Scalar**: `Fr` (the prime field of order `r`), used for shares `s_i` and the (notional) master secret `MSK`.

Hash-to-curve uses RFC 9380 with the suite tag `BLS12381G1_XMD:SHA-256_SSWU_RO_` and the CBSS domain `cbss/ibe/v1`. Symmetric encryption is **AES-256-GCM**; key derivation is **HKDF-SHA-256**.

5.2 Identity Construction

For a secret version (`account`, `key_hash`, `version`, `wrap_epoch`):

```
canonical(SecretId) = account || key_hash           -- 52 bytes
aad                 = canonical(SecretId)
                    || u64_le(version)
                    || u64_le(wrap_epoch)           -- 68 bytes total
I                   = hash_to_G1(aad, domain = "cbss/ibe/v1")
```

`wrap_epoch` is the `committee_epoch` at the moment the secret version was wrapped. It is **immutable** once set on `SecretVersion` and is the load-bearing field that ties the identity to the historical share polynomial under which it was encrypted.

5.3 Encryption (Owner Side)

Given MPK, identity I , plaintext DEK dek (256 bits, fresh CSPRNG), and the same aad :

```

gt          = e(I, MPK)                                -- pairing in Gt
ikm         = serialize(gt)                             -- canonical Gt encoding
K           = HKDF-SHA-256(salt = "cbss/ibe/v1",
                        ikm,
                        info = "cbss/ibe/v1" || aad,
                        L = 32)
nonce       ← CSPRNG(12)
WrappedDek  = AES-256-GCM(key = K,
                        nonce = nonce,
                        aad = aad,
                        plaintext = dek)

```

The `WrappedDek` (ciphertext, nonce, aad) is stored on chain. The plaintext payload is separately AES-256-GCM-encrypted under dek and the same aad and uploaded to a CBFS volume; the resulting `cbfs_pointer` is also stored on chain.

5.4 DKG (Once Per Account / Per-Secret Override)

A committee of n proxies runs a Pedersen-style DKG with verifiable secret sharing. Each proxy j (with index $1 \leq j \leq n$):

1. Samples a degree- $(t-1)$ polynomial $f_j(x)$ with t random coefficients.
2. Publishes Round 1: $commitments_j = (g_2 \cdot f_j(0), g_2 \cdot coeff_j[1], \dots, g_2 \cdot coeff_j[t-1])$.
3. Sends Round 2: each recipient k receives $f_j(k)$ (HPKE-encrypted to recipient k 's X25519 key registered in `RegisterCbssProxy`).
4. Each recipient verifies their share against the public commitments; mismatches yield slashable evidence.

After all rounds verify, the proxy's share is $s_k = \sum_j f_j(k)$ and the global MPK is the sum of every dealer's `commitments_j[0]`. The aggregate VSS commitments at index i evaluate to the same polynomial in G_2 , so the chain can verify any partial signature against them.

The DKG is bootstrapped by a `RequestAccountDkg` (default-path, owner-paid bond) or by `SetSecret` with `committee_override` (override-path, owner-paid bond). It commits via

RotateCommittee (signed by $\geq t$ of the new committee) when the ceremony completes; if it does not complete within `DKG_FINALIZE_BLOCKS`, anyone may call **ExpireDkgPending** to refund the requester (minus a small slash) and clear the pending record.

5.5 Partial Sign and Threshold Combine (Runner Side)

A proxy holding share `s_i` for committee_epoch `e`, on receiving a runner-signed **PartialSignRequest**, validates the request (§6.2), reads `wrap_epoch` from the on-chain **SecretVersion**, derives `I = hash_to_G1(aad)`, and computes:

```
 $\sigma_i = s_i \cdot I$            -- a G1 point, 48 bytes compressed
```

The proxy signs (`request_hash`, `proxy_id`, `sigma_i`, `committee_epoch_at_serve`, `served_at_block`) under its operator key (domain `cbss/partial-sign-response/v1`) and returns the response.

The runner verifies each σ_i via the pairing equation:

```
 $e(\sigma_i, G2\_gen) == e(I, S_i)$ 
```

where `S_i` is proxy `i`'s aggregate VSS commitment looked up on chain. With `t` valid partials, the runner computes Lagrange coefficients $\lambda_i(0)$ over the chosen index set and combines:

```
 $\sigma = \sum (\lambda_i(0) \cdot \sigma_i)$  over the t selected indices
```

By bilinearity, $\sigma = MSK \cdot I$.

5.6 Decryption (Runner Side)

With the threshold signature σ , the runner derives the same `K` the owner used:

```
gt      = e( $\sigma$ , G2_gen)
ikm     = serialize(gt)
K       = HKDF-SHA-256(salt = "cbss/ibe/v1",
                      ikm,
                      info = "cbss/ibe/v1" || aad,
                      L = 32)
dek     = AES-256-GCM-decrypt(K, nonce, aad, WrappedDek.ciphertext)
```

The bilinearity of the pairing guarantees the equality $e(\sigma, G2_gen) = e(MSK \cdot I, G2_gen) = e(I, MSK \cdot G2_gen) = e(I, MPK)$, so the runner's K matches the owner's K exactly. The runner then fetches the CBFS object at `cbfs_pointer`, AES-256-GCM-decrypts under (dek, aad) , zeroes both `dek` and the plaintext after substitution.

5.7 Reshare via Proactive Secret Sharing

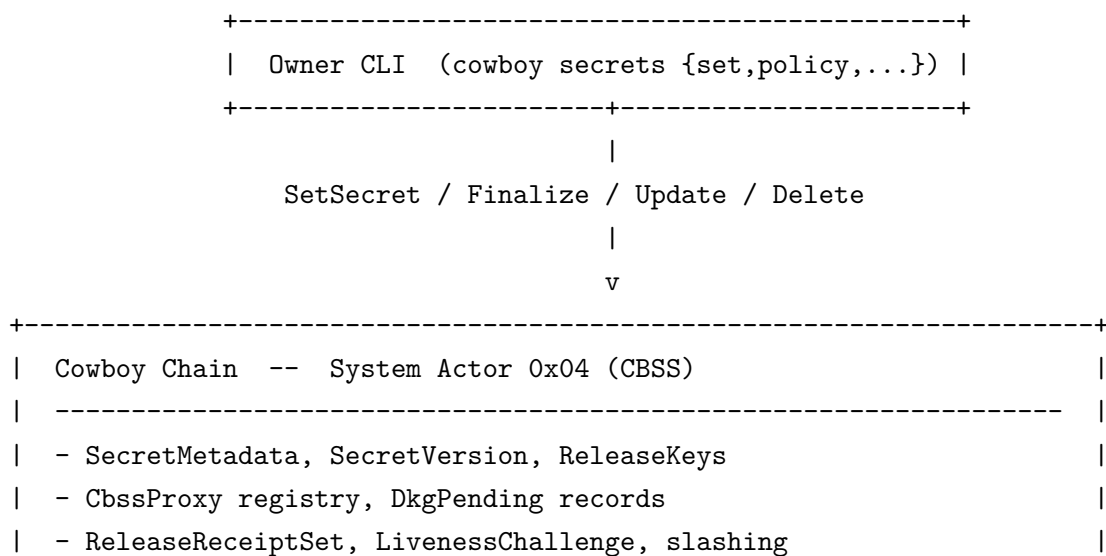
A reshare replaces all proxy shares with fresh shares for the same MSK. Each old-committee participant j (with old share s_j) selected as a dealer constructs a polynomial:

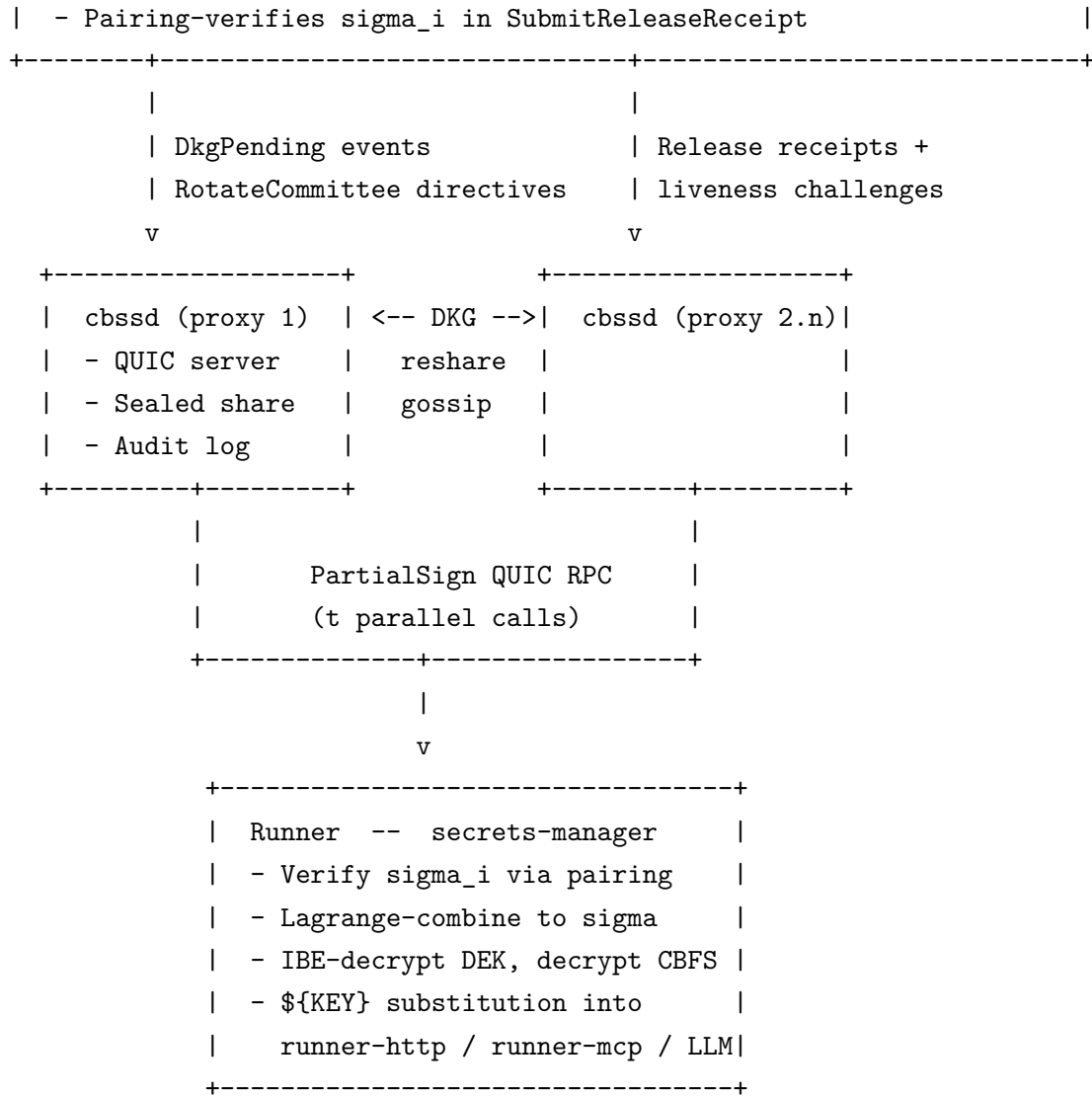
```
g_j(0)    = s_j · L_j          -- L_j is the Lagrange coefficient at zero for the chosen
g_j(x)    = g_j(0) + r_j(x)   -- r_j is a fresh degree-(t-1) polynomial with r_j(0) = 0
```

Each new-committee recipient k receives $g_j(k)$ from every dealer (HPKE-encrypted), verifies it against published commitments, and sets its new share to $s'_k = \sum_j g_j(k)$. Because $\sum_j g_j(0) = \sum_j (s_j \cdot L_j) = MSK$, the new shares lie on a polynomial whose constant term is MSK; MPK is preserved.

The chain enforces `args.new_mpk == old.mpk` and `args.new_committee_epoch == old.committee_epoch + 1` on `RotateCommittee`. It also retains the prior committee's commitments in `prior_committee_epochs` for `RESHARE_GRACE_BLOCKS`, so receipts under the prior epoch remain acceptable during the grace window.

6 System Architecture





6.1 On-Chain Component (System Actor 0x04)

The CBSS system actor occupies address 0x04 (per Technical Whitepaper §9). It owns the storage shapes in §2, dispatches the 14 system instructions in §3, and emits the events in §4. It performs all pairing verifications for `SubmitReleaseReceipt` and validates committee co-signatures for `RotateCommittee`. It does **not** hold any plaintext, any DEK, or any share scalar.

6.2 Off-Chain Component (cbssd Daemon)

Each CBSS proxy operator runs `cbssd`, a Tokio-based binary that:

- Maintains an `secp256k1` operator identity (registered on chain) and an `X25519` HPKE identity (registered on chain) for DKG/reshare share encryption.
- Listens on a QUIC endpoint multiplexing four logical streams (`PartialSign`, `Health`, `DkgRound`, `ReshareRound`) on a single port (default 9601).
- Watches the chain for `DkgPending` records and `RotateCommittee` directives that name this proxy and orchestrates the corresponding ceremonies.
- Stores per-scope sealed shares at rest, sealed via AES-256-GCM under a key derived from the operator key.
- On `PartialSign`, validates the request against current head, computes $\sigma_i = s_i \cdot I$, signs the response, and (after a 0–10 block jitter) submits a `SubmitReleaseReceipt` transaction.
- Maintains a 24-hour replay-bloom keyed on (`release_id`, `proxy_id`) so the same release is never partial-signed twice by the same proxy.
- Submits `LivenessChallengeResponse` for any chain-recorded challenge against this proxy.

`cbssd` is a separate process from `cbfsd` and `runnerd`; operators MAY co-locate but MUST run separate stake pools.

6.3 Runner-Side Client (`secrets-manager` Crate)

The runner-side client is a Rust crate consumed by `runner-node`, `runner-http`, `runner-mcp`, and `runner-llm`. It exposes a single trait:

```
[async_trait]
pub trait SecretsClient: Send + Sync {
    async fn fetch_for_op(
        &self,
        ctx: &JobContext,
        actor: ActorAddress,
        keys: &[SecretRef],
    ) -> Result<SecretBundle, SecretsError>;
}
```

Production implementations fan out partial-sign QUIC RPCs to `t` of `n` committee proxies, verify each σ_i via the pairing equation, Lagrange-combine into σ , IBE-decrypt the DEK, fetch and AES-decrypt the CBFS payload, and return a `SecretBundle` whose entries are `Zeroizing<Vec<u8>>`. The bundle is consumed by `KEY` substitution helpers (`substitute_str`, `substitute_json`, `substitute_string_map`) wired into `runner-http/mcp/llm` at request-construction time and dropped (zeroized) when the request completes.

7 1. Identifiers and Canonical Forms

1.1 **Account.** A `secp256k1`-derived 20-byte Cowboy address (Technical Whitepaper §1.1).

1.2 **Key name.** UTF-8, ≤ 128 bytes. The hash on chain is `key_hash = keccak256(account || key_name)`. Plaintext key names MUST NOT appear in storage paths or events — only `key_hash`.

1.3 **SecretId.** (`account`, `key_hash`). Canonical encoding is the 52-byte concatenation `account || key_hash`.

1.4 **Versioning.** Versions are monotonic `u64` integers per `SecretId`. Quotas: `MAX_SECRETS_PER_ACCOUNT = 1024`, `MAX_VERSIONS_PER_SECRET = 256`, `MAX_ACL_ACTORS = 64`.

1.5 **Release nonce.** Per-`fetch_for_op` 32-byte CSPRNG. Used to derive `release_id = keccak256(canonical(SecretId) || u64_le(version) || actor || runner_id || job_id || release_nonce)`.

1.6 **Wrap epoch.** The `committee_epoch` of the release key at the moment a secret version is wrapped. Stored on `SecretVersion`. **IMMUTABLE** once set.

1.7 **Serve epoch.** The `committee_epoch` the runner pinned in the request body when fanning out partial-sign RPCs. Equal to the `committee_epoch` of the release key at that block height. May differ from `wrap_epoch` after one or more reshares.

1.8 **Domain separators.** All hashes for signing or hash-to-curve MUST be prefixed with one of:

- `cbss/ibe/v1` — identity I and IBE key derivation.
- `cbss/release-request/v1` — runner signature over `ReleaseRequestBody`.
- `cbss/partial-sign-response/v1` — proxy signature over (`request_hash`, `proxy_id`, σ_i , `committee_epoch_at_serve`, `served_at_block`).
- `cbss/rotate-committee/v1` — committee co-signature over `RotateCommitteeArgs`.
- `cbss/liveness-challenge/v1` — runner signature over `SubmitLivenessChallengeArgs.runner_request`.
- `cbss/share-storage/v1` — at-rest seal of operator share material.
- `cbss/dkg-share-hpke/v1` — HPKE info for inter-proxy DKG share envelopes.

8 2. Storage Shapes

`0x04` storage uses path-prefixed keys; values are JSON unless noted. All hashed where the secret key name would otherwise appear.

secret:{account_hex}:{key_hash_hex}	→ SecretMetadata
secret_version:{account_hex}:{key_hash_hex}:{version_be8}	→ SecretVersion
secret_version_tombstone:{...}:{version_be8}	→ SecretVersionTombstone
account_release_key:{account_hex}	→ AccountReleaseKey
secret_release_key:{key_hash_hex}:{version_be8}	→ SecretReleaseKey
cbss_proxy:{proxy_id_hex}	→ CbssProxy
cbss_proxy_index_by_operator:{operator_hex}	→ ProxyId
release_receipt:{release_id_hex}	→ ReleaseReceiptSet
liveness_challenge:{challenge_id_hex}	→ LivenessChallenge
liveness_challenge_index:{release_id_hex}:{proxy_id_hex}	→ ChallengeId
dkg_pending:{scope_serialized}	→ DkgPending

The principal value shapes:

```

struct SecretMetadata {
    id: SecretId,
    latest_version: u64,
    active_versions: Vec<u64>,
    created_at: BlockHeight,
    last_updated_at: BlockHeight,
}

struct SecretVersion {
    version: u64,
    pending: bool, // true between override SetSecret and
↪ FinalizeSecretVersion
    recipient: ReleaseKeyRef, // Account(_) or SecretSpecific[...]
    policy: SecretPolicy, // actors allowlist + tee_required
    wrap_epoch: Option<u64>, // immutable once set; None iff pending
    cbfs_pointer: Option<CbfsPointer>, // None iff pending
    wrapped_dek: Option<WrappedDek>, // None iff pending
    created_at: BlockHeight,
}

struct AccountReleaseKey {
    account: Address,
    mpk: G2Bytes,
    committee: Vec<ProxyId>,
    threshold: u8,
    committee_epoch: u64,
    last_reshare_block: BlockHeight,
    vss_commitments: Vec<G2Bytes>,

```

```

    prior_committee_epochs: Vec<PriorCommitteeEpoch>, // capped at
↪ MAX_PRIOR_COMMITTEE_EPOCHS
}

struct PriorCommitteeEpoch {
    epoch: u64,
    committee: Vec<ProxyId>,
    vss_commitments: Vec<G2Bytes>,
    expires_at: BlockHeight, // last_resshare_block + RESHARE_GRACE_BLOCKS
}

struct ReleaseReceiptSet {
    quorum_epoch: u64, // pinned to first accepted receipt's
↪ serve_epoch
    threshold: u8,
    entries: Vec<ReleaseReceipt>, // capped at threshold
}

```

The invariant `pending == true ⇔ wrap_epoch.is_none() ⇔ cbfs_pointer.is_none() ⇔ wrapped_dek.is_none()` MUST hold for every `SecretVersion`. Once `wrap_epoch` is set on a non-pending version, it MUST NOT change.

9 3. System Instructions (Opcodes 68–81)

The CBSS system actor implements 14 instructions, encoded as `SystemInstruction` variants per Technical Whitepaper §3 dispatch.

Op	Name	Caller	Purpose
68	<code>SetSecret</code>	Owner	Set or override-intent a secret version
69	<code>UpdateSecretPolicy</code>	Owner	Mutate policy only; never re-wraps
70	<code>DeleteSecretVersion</code>	Owner	Tombstone one version
71	<code>DeleteSecret</code>	Owner	Tombstone all versions; drop release key if scoped

Op	Name	Caller	Purpose
72	RegisterCbssProxy	Operator	Stake and register a proxy with HPKE pubkey
73	DeregisterCbssProxy	Operator	Initiate unbond
74	RotateCommittee	Committee co-sign	Commit a DKG or reshare result
75	SlashCbssProxy	Anyone (with evidence)	Apply per-class slash + suspend
76	SubmitReleaseReceipt	Proxy	Anchor a partial signature on chain, earn fee
77	RequestAccountDkg	Owner	Bond a default-path account DKG
78	FinalizeSecretVersion	Owner	Attach <code>cbfs_pointer</code> + <code>wrapped_dek</code> after override DKG commits
79	SubmitLivenessChallenge	Owner	Bond a non-response challenge against a proxy
80	LivenessChallengeResponse	Proxy operator	Respond to a chain-recorded challenge
81	ExpireDkgPending	Anyone	Permissionless cleanup of expired DKG bonds

3.1 SetSecret (default path). Owner has an existing `AccountReleaseKey`. Args carry `(key_hash, recipient = Account(self), policy, wrap_epoch, cbfs_pointer, wrapped_dek)`. Chain checks `wrap_epoch == release_key.committee_epoch` and `wrapped_dek.aad == aad_for(secret_id, version, wrap_epoch)`; writes a non-pending `SecretVersion`.

3.2 SetSecret (override path). Args carry (key_hash, recipient = SecretSpecific{...}, policy, committee_override = {t, n, preferred_proxies?}) and NO ciphertext. Chain validates the committee, locks DKG_BOND from the owner, enqueues a DkgPending record, and writes a *pending* SecretVersion with wrap_epoch = None.

3.3 FinalizeSecretVersion. Once the per-secret DKG commits via RotateCommittee, the owner calls FinalizeSecretVersion(key_hash, version, wrap_epoch, cbfs_pointer, wrapped_dek). Chain checks wrap_epoch == new_release_key.committee_epoch, wrapped_dek.aad == aad_for(secret_id, version, wrap_epoch), clears pending, and attaches the ciphertext.

3.4 UpdateSecretPolicy. Pure metadata. Chain replaces policy.actors and/or policy.tee_required on the latest non-pending version. No WrappedDek is touched. ACL changes propagate at chain finality plus PROXY_CACHE_TTL_BLOCKS.

3.5 DeleteSecretVersion / DeleteSecret. Writes a SecretVersionTombstone so subsequent reads return SecretVersionDeleted rather than SecretNotFound. DeleteSecret on an override-scoped secret also drops the SecretReleaseKey.

3.6 RegisterCbssProxy. Args: (operator_pubkey, hpke_pubkey, network_addr, stake). Chain stakes stake $\geq$ MIN_PROXY_STAKE, sets eligible_after = block_height + PROXY_SOAK_PERIOD_BLOCKS, and inserts into the proxy index. The proxy is NOT VRF-eligible until eligible_after is reached.

3.7 DeregisterCbssProxy. Sets unbond_at = block_height + PROXY_UNBOND_BLOCKS. The proxy is removed from new committee selections immediately; existing committees containing this proxy MUST trigger reshare.

3.8 RotateCommittee. Args: (scope, new_committee, new_committee_epoch, new_mpk, vss_commitments, new_committee_sigs, prior_committee_sigs). Chain computes rotate_committee_hash and verifies:

- $\geq t$ signatures from the new committee under domain cbss/rotate-committee/v1.
- If updating an existing release key: $\geq t$ signatures from the *prior* committee, and new_mpk == old.mpk, and new_committee_epoch == old.committee_epoch + 1.
- Threshold reused from old.threshold (or, on first DKG, from DkgPending.threshold).
- vss_commitments valid: vss_commitments[0] == new_mpk, length = threshold, all decode to non-identity G2 points.

On success, the prior committee's epoch is appended to prior_committee_epochs, capped at MAX_PRIOR_COMMITTEE_EPOCHS and pruned of expired entries.

3.9 SlashCbssProxy. Args: (proxy_id, fault_class $\in$ {InvalidPartialReencrypt, DkgSabotage, Equivocation}, evidence). The chain re-derives the offence from evidence,

charges the per-class slash percentage (5 % / 5 % / 50 %), and suspends the proxy. Slashing happens at most once per (proxy, fault) pair within a sliding window.

3.10 `SubmitReleaseReceipt`. Args: (`runner_request`: `SignedReleaseRequest`, `proxy_id`, `sigma_i`, `committee_epoch_at_serve`, `served_at_block`, `proxy_sig`). Chain validates:

- `body.request_block` $\in$ [`head` - `REQUEST_FRESHNESS_BLOCKS`, `head`].
- `args.committee_epoch_at_serve` == `body.serve_epoch`; both match a current or grace-window release key view.
- `runner_sig` recovers to `body.runner_id` under `cbss/release-request/v1`.
- `proxy_sig` recovers to `proxy.operator` under `cbss/partial-sign-response/v1`.
- `SecretVersion` exists, is non-pending, has `recipient` == `body.recipient`, and yields the `wrap_epoch` for identity derivation.
- `policy.actors` (if set) contains `body.actor`; `policy.tee_required` validates against the runner's CIP-23 attestation at submission-time current head.
- The pairing equation $e(\sigma_i, G2_gen) == e(I, S_i)$ holds, with $S_i = \sum_k vss_commitments[k] \cdot proxy_index^k$ evaluated from the release-key view.
- The (`release_id`, `proxy_id`) pair has not already been recorded; the `ReleaseReceiptSet` has not yet reached `threshold` entries.

On success, the receipt is appended to the set, the actor is debited `RELEASE_FEE_PER_RECEIPT`, and the proxy is credited the same. The chain emits `cbss.release.receipt_recorded` per receipt and `cbss.secret.released` exactly once when the *t*-th receipt lands.

3.11 `SubmitLivenessChallenge`. Args: (`runner_request`, `proxy_id`, `challenge_sig`). Chain enforces `block_height` $\geq$ `body.request_block` + `MIN_CHALLENGE_DELAY_BLOCKS` and `block_height` $\leq$ `body.request_block` + `LIVENESS_CHALLENGE_MAX_AGE_BLOCKS`. `challenge_sig` MUST recover to `body.runner_id` under `cbss/liveness-challenge/v1`. The chain checks for an existing receipt: if present, the challenge admits as `Resolved` immediately and the bond transfers to the proxy. Otherwise it admits as `Pending` with `deadline` = `block_height` + `LIVENESS_RESPONSE_BLOCKS`.

3.12 `LivenessChallengeResponse`. Proxy operator submits with `reason` $\in$ {`InvalidRequest`, `RateLimited`, `NeverReceived`}. Chain transitions the challenge to `Unverifiable` (terminal), refunds the bond to the runner, and leaves the proxy's health neutral.

3.13 **Liveness deadline**. A challenge that reaches `block_height` > `deadline` without a response transitions to `Unanswered`, refunds the bond to the runner, and decrements the proxy's `health.score_bps`. The transition is performed permissionlessly via an `ExpireLivenessChallenge` instruction (mirroring `ExpireDkgPending`); a per-block tick is NOT used.

3.14 ExpireDkgPending. Permissionless cleanup of expired DKG bonds. Refunds the requester `DKG_BOND - DKG_TIMEOUT_SLASH`, pays the caller a small fixed reward, and clears the `DkgPending` record. For override-path expiries, the corresponding pending `SecretVersion` is also erased (no tombstone — the slot is reusable on retry).

10 4. Events

The CBSS system actor emits the following events (topics carry the listed key tags as keccak-prefix bytes):

Topic	Body	When
<code>cbss.secret.created</code>	<code>(secret_id, version)</code>	After a non-pending <code>SetSecret</code> or <code>FinalizeSecretVersion</code> writes a usable version
<code>cbss.secret.policy_updated</code>	<code>(secret_id, version)</code>	After <code>UpdateSecretPolicy</code>
<code>cbss.secret.deleted</code>	<code>(secret_id, version, reason)</code>	After <code>DeleteSecretVersion</code> or <code>DeleteSecret</code>
<code>cbss.proxy.registered</code>	<code>proxy_id</code>	After <code>RegisterCbssProxy</code>
<code>cbss.proxy.deregistered</code>	<code>proxy_id</code>	After <code>DeregisterCbssProxy</code>
<code>cbss.proxy.slashed</code>	<code>(proxy_id, fault_class)</code>	After <code>SlashCbssProxy</code>
<code>cbss.committee.rotated</code>	<code>scope</code>	After <code>RotateCommittee</code> succeeds
<code>cbss.dkg.requested</code>	<code>account_or_scope</code>	After <code>RequestAccountDkg</code> or override <code>SetSecret</code>
<code>cbss.dkg.expired</code>	<code>scope</code>	After <code>ExpireDkgPending</code>
<code>cbss.release.receipt_recorded</code>	<code>release_id</code>	Per accepted <code>SubmitReleaseReceipt</code>
<code>cbss.secret.released</code>	<code>release_id</code>	Exactly once when the <code>t</code> -th receipt lands
<code>cbss.liveness.challenge</code>	<code>challenge_id</code>	After <code>SubmitLivenessChallenge</code> admits

Topic	Body	When
<code>cbss.liveness.response</code>	<code>challenge_id</code>	After <code>LivenessChallengeResponse</code>
<code>cbss.liveness.expired</code>	<code>challenge_id</code>	After <code>ExpireLivenessChallenge</code>

11 5. The Release Path (Normative)

5.1 Construction. The runner's `secrets-manager` client constructs:

```

release_nonce    ← CSPRNG(32)
request_block    = chain_head_height
serve_epoch      = release_key.committee_epoch (at chain_head_height)
body             = ReleaseRequestBody { secret_id, version, actor, runner_id,
                                     recipient, job_id, release_nonce,
                                     request_block, serve_epoch }
runner_sig       = ECDSA-sign(runner_sk,
                              keccak256("cbss/release-request/v1" || encode(body)))
request          = SignedReleaseRequest { body, runner_sig }

```

5.2 Fanout. The runner sends `request` to `t` selected proxies in parallel via QUIC. Selection prefers lowest known latency; ties are randomised. Sub-RTT is expected; the runner imposes a deadline of `PARTIAL_SIGN_RUNNER_DEADLINE_MS` (default 1500).

5.3 Proxy validation. On receipt of a `PartialSignRequest`, the proxy enforces, in order:

1. `body.request_block` $\in$ `[head - REQUEST_FRESHNESS_BLOCKS, head]`.
2. `runner_sig` recovers to `body.runner_id` under `cbss/release-request/v1`.
3. Runner currently assigned to `body.job_id` (chain client, current head).
4. `actor` manifest declares `body.secret_id.key_hash` (chain client, current head).
5. `policy.actors` (if set) contains `body.actor`; `policy.tee_required` validates if set.
6. Proxy is a member of the referenced release key's current committee.
7. `body.serve_epoch` `==` `release_key.committee_epoch`.
8. (`release_id`, `proxy_id`) not in 24-hour replay bloom.
9. Per-actor rate limit not exceeded.

5.4 Signing. The proxy decrypts its share `s_i`, reads `wrap_epoch` from the on-chain `SecretVersion`, computes `I = hash_to_G1(aad)`, computes `σ_i = s_i · I`, signs the response payload, updates the bloom and rate-limit windows, and returns. CPU-heavy steps run on `tokio::task::spawn_blocking`.

5.5 Combine. The runner verifies each `σ_i` against the on-chain `S_i` (lazy chain reads cached for `PROXY_CACHE_TTL_BLOCKS`), Lagrange-combines `t` valid partials into `σ`, and IBE-decrypts the DEK. The DEK is wrapped in `Zeroizing<u8; 32>` for the duration of the operation.

5.6 Payload. The runner fetches the CBFS object at `cbfs_pointer` (CapToken-authenticated per CIP-9), AES-256-GCM-decrypts under `(dek, aad)`, inserts the resulting `Zeroizing<Vec<u8>>` into the `SecretBundle` keyed on the original `KeyName`, and zeroes the DEK.

5.7 Substitution. Runner-host modules consume the bundle:

- **runner-http:** substitutes `${KEY}` in URL, headers (including `Authorization`), and request body bytes.
- **runner-mcp:** substitutes `${KEY}` in MCP tool arguments (JSON-aware) and, for stdio MCP servers, spawns the child via `tokio::process::Command::env_clear().envs(env)` so the parent process environment is never mutated.
- **runner-llm:** only when server-side tool plumbing is active; otherwise rejects with `SecretsNotApplicableHere`.

5.8 Drop. When the request completes (or panics), the bundle drops; entries are zeroized via `Zeroizing<Vec<u8>>::drop`. PVM Python actors are blocked from `os.environ` reads of any declared secret key (sandbox install-time deny list, Technical Whitepaper §3.1).

5.9 Receipts. Asynchronously and independently, each of the `t` participating proxies submits `SubmitReleaseReceipt` with 0–10 blocks of jitter. The runner does NOT submit. Per-proxy events fire as receipts land; the aggregate `cbss.secret.released` fires once, deterministically, when the `t`-th lands.

12 6. The DKG and Reshare Path (Normative)

6.1 DKG trigger. A `DkgPending` record exists for `(scope, threshold, deadline, requester)`. The committee is either selected by the chain (`select_eligible_proxies`) or named explicitly via `committee_override.preferred_proxies`. Each member discovers the others' `network_addr` and `hpke_pubkey` from the chain.

6.2 DKG ceremony. The committee establishes a full-mesh QUIC topology and runs the protocol of §0 cryptographic construction. Round-2 share envelopes are HPKE-encrypted to each

recipient's X25519 key. Each recipient validates received shares against the public commitments; mismatches yield `DkgSabotage` evidence eligible for `SlashCbssProxy`.

6.3 DKG commit. When all participants finalize, the lowest-`ProxyId` member submits `RotateCommittee` on behalf of the committee, signed by $\geq \text{threshold}$ members. The chain refunds the requester's `DKG_BOND` on success. If the ceremony fails to commit before `DKG_FINALIZE_BLOCKS`, anyone may submit `ExpireDkgPending`; the requester is refunded `DKG_BOND - DKG_TIMEOUT_SLASH` and the cleanup-caller earns `EXPIRE_DKG_REWARD` (default 1 token unit).

6.4 Reshare trigger. Block-height scheduler (`now` $\geq \text{last_reshare_block} + \text{RESHARE_INTERVAL_BLOCKS}$), churn detection (committee size near `t + RESHARE_SAFETY_MARGIN`), or owner-paid `RotateCommittee` request.

6.5 Reshare ceremony. A subset of $\geq t$ old-committee members run the PSS protocol of §0 cryptographic construction. New-committee members verify each dealer's contribution against the on-chain old commitments. On success, the new committee submits `RotateCommittee` co-signed by $\geq \text{threshold}$ of itself AND $\geq \text{threshold}$ of the prior committee.

6.6 Old-share zeroization. Old-committee members MUST zeroize their old `SealedShare` files atomically with observing the on-chain `RotateCommittee` event. The zeroization MUST be recorded in the operator audit log. Failure to zeroize is an operational fault, not a slashable offence (the share is still useful only for `committee_epoch` values within the reshare grace window).

6.7 Grace window. For `RESHARE_GRACE_BLOCKS` after a rotation, receipts under the prior epoch remain acceptable: the chain looks up the receipt's `committee_epoch_at_serve` first against the current release key, then against `prior_committee_epochs`. The grace window is capped at `MAX_PRIOR_COMMITTEE_EPOCHS` simultaneously retained epochs.

13 7. Slashing

CBSS slashes for three on-chain-provable fault classes only. All other operational issues (non-response, slow proxies, miscalibrated stake) flow through the liveness-challenge audit trail and CIP-12 governance, not through automatic slashing.

Fault	Penalty	Evidence
<code>InvalidPartialReencrypt</code>	5 % of stake + suspend	A <code>SubmittedReleaseReceipt</code> whose σ_i fails the pairing check $e(\sigma_i, G2_gen) == e(I, S_i)$ for the proxy's index. The chain re-derives I from the body, looks up S_i from the release key, and re-runs the pairing.
<code>DkgSabotage</code>	5 % of stake + suspend	A round-2 message whose recipient share is inconsistent with the dealer's published round-1 commitments. The chain verifies $g_2 \cdot \text{share_scalar} != \sum \text{commitments}[k] \cdot \text{recipient_index}^k$.
<code>Equivocation</code>	50 % of stake + suspend	Two distinct <code>PartialSignResponse</code> payloads carrying identical <code>request_hash</code> but different σ_i . Both signatures must verify under the same proxy operator pubkey.

The handler is `SlashCbssProxy`. The chain de-duplicates evidence: a given (proxy, fault, evidence_keccak) pair MUST NOT slash twice within a sliding window of `SLASH_WINDOW_BLOCKS`. Plaintext leaks (off-chain disclosure) and persistent low health are NOT auto-slash; they route through CIP-12 Tier 1 governance via `ForcedDeregisterCbssProxy`.

14 8. Gas Pricing

CBSS instructions consume gas under both meters (Cycles + Cells, Technical Whitepaper §4 / §17.1). Pricing per CIP-24 §4.1:

Instruction	Cycles	Cells
<code>SetSecret</code> (default)	200,000	8 + <code>len(wrapped_dek.ciphertext)</code> + <code>len(cbfs_pointer)</code>
<code>SetSecret</code> (override)	350,000	8 + <code>len(committee_override)</code>
<code>FinalizeSecretVersion</code>	250,000	8 + <code>len(wrapped_dek.ciphertext)</code> + <code>len(cbfs_pointer)</code>
<code>UpdateSecretPolicy</code>	80,000	8 + <code>len(policy)</code>
<code>DeleteSecretVersion</code>	60,000	0
<code>DeleteSecret</code>	$60,000 + 20,000 \times$ <code>latest_version</code>	0
<code>RegisterCbssProxy</code>	150,000	8 + <code>len(network_addr)</code> + 32
<code>DeregisterCbssProxy</code>	80,000	0
<code>RotateCommittee</code>	$1,200,000 + 80,000 \times n +$ <code>threshold</code>	8 + $96 \times (1 + \text{threshold})$ + $32 \times n + 65 \times (\text{sigs})$
<code>SlashCbssProxy</code>	800,000	8 + <code>len(evidence)</code>
<code>SubmitReleaseReceipt</code>	600,000	8 + 48 + 65 + 32
<code>RequestAccountDkg</code>	200,000	8
<code>SubmitLivenessChallenge</code>	250,000	8 + <code>len(runner_request)</code> + 65
<code>LivenessChallengeResponse</code>	180,000	8 + <code>len(evidence)</code> + 65
<code>ExpireDkgPending</code>	100,000	0

Cycle counts are dominated by pairing operations on `SubmitReleaseReceipt` and by committee-signature recovery on `RotateCommittee`. Cell counts are exact byte costs of the encoded fields.

15 9. Liveness, Health, and Slashing Fairness

9.1 Liveness challenge bond. `LIVENESS_CHALLENGE_BOND` (default 100 token units). Locked on `SubmitLivenessChallenge`, refunded on `Unverifiable` or `Unanswered`, transferred to the proxy on `Resolved`.

9.2 Health score. Each `CbssProxy` carries a `health.score_bps` (max 10,000). Decrement by `LIVENESS_UNANSWERED_DECREMENT_BPS` on each `Unanswered`, by 2,500 on each successful slash. Increased slowly over time (`HEALTH_RECOVERY_BPS_PER_BLOCK`) up to the cap. A proxy with `score_bps < HEALTH_GOVERNANCE_FLOOR_BPS` for `GOVERNANCE_REVIEW_BLOCKS` (≈ 30 days) flags an automatic CIP-12 Tier 1 `ForcedDeregisterCbssProxy` proposal.

9.3 Bond economics. Spam against responsive proxies is expensive: every challenge costs `LIVENESS_CHALLENGE_BOND` and resolves immediately to the proxy if the proxy already produced a receipt. The constraint `MIN_CHALLENGE_DELAY_BLOCKS > REQUEST_FRESHNESS_BLOCKS` ensures honest receipts have already landed (or can no longer land) before a challenge is admissible.

16 10. Owner-Side Flows (CLI)

10.1 Default-path set:

```
cowboy secrets set OPENAI_API_KEY <value>
```

The CLI looks up the owner's `AccountReleaseKey`. If absent, it submits `RequestAccountDkg` and waits for `cbss.committee.rotated`. It generates a fresh DEK, computes `aad`, derives `K = HKDF(serialize(e(I, MPK)), "cbss/ibe/v1" || aad)`, AES-encrypts the value to a CBFS object, AES-encrypts the DEK under `K` into a `WrappedDek`, and submits `SetSecret`.

10.2 Override-path set:

```
cowboy secrets set STRIPE_LIVE_KEY <value> --committee-override --threshold 5/9
```

The CLI submits `SetSecret(committee_override = {t=5, n=9})` with no ciphertext and a recipient of `SecretSpecific{...}`. It waits for the per-secret DKG to commit, then submits `FinalizeSecretVersion` with the wrapped DEK and CBFS pointer.

10.3 Policy update:

```
cowboy secrets policy OPENAI_API_KEY --add-actor 0x... --remove-actor 0x...
```

Pure metadata; no DEK touched. Effect propagates within `finality + PROXY_CACHE_TTL_BLOCKS`.

10.4 Versioning and deletion:

```
cowboy secrets versions OPENAI_API_KEY
cowboy secrets delete-version OPENAI_API_KEY --version 3
cowboy secrets delete OPENAI_API_KEY
```

10.5 Operator flows:

```
cowboy proxy register --stake 10000 --network quic://1.2.3.4:9601
cowboy proxy status
cowboy proxy reshare-status
```

17 11. SDK Integration

11.1 **Manifest.** Actors declare in their `ActorManifest`:

```
{
  "secrets": {
    "read": {
      "keys": [
        { "type": "OwnAccount", "key": "OPENAI_API_KEY" },
        { "type": "CrossAccount", "account": "0x...", "key": "SLACK_API_KEY" }
      ]
    }
  }
}
```

Manifest upgrades MAY shrink `secrets.read.keys` but MUST NOT expand them; the entitlement diff check is enforced in the existing CIP-13 upgrade path.

11.2 **Python SDK.** Runner ops accept a `secrets=[KEYS]` kwarg:

```
result = runner.http(
    method="POST",
    url="https://api.openai.com/v1/chat/completions",
    headers={"Authorization": "Bearer ${OPENAI_API_KEY}"},
    json=payload,
    secrets=["OPENAI_API_KEY"],
)
```

`secrets=[...]` MUST be a subset of the manifest's `secrets.read.keys`; otherwise the SDK raises `SecretsNotDeclaredInManifest`. `runner.llm(..., secrets=[...])` is accepted only when server-side tool plumbing is active and raises `SecretsNotApplicableHere` otherwise.

11.3 PVM environment isolation. The PVM Python sandbox installs a deny list at handler start covering every key declared in the actor's `secrets.read` entitlement. Any `os.environ.get(KEY)` for a declared key returns `None`, even if the runner-host process actually holds the value. Logging adapters scrub strings matching active bundle values from spans and trace events as a defence-in-depth measure.

18 12. Security Considerations

12.1 Threshold IBE assumption. Security reduces to the Computational co-Diffie-Hellman assumption on BLS12-381 plus the random-oracle model for `hash_to_G1`. Standard threshold-BLS analysis (Boneh, Lynn, Shacham 2001; Boldyreva 2003) covers the partial signature combine. The IBE construction matches Boneh-Franklin's BasicIdent transformed to threshold form via a vetted t-of-n DKG; we follow the drand `tlock` paper's specialization with our chain-anchored identity tuple.

12.2 DKG bias. A naive Pedersen DKG can be biased by a malicious last-publisher. CBSS uses the bias-free Gennaro-Jarecki-Krawczyk-Rabin variant via the `blsful` crate, which adds a complaint phase and qualified-set logic. Implementations MUST use `blsful::dkg` (or a binary-equivalent) rather than rolling a custom Pedersen DKG.

12.3 Reshare insider rotation. PSS preserves MSK; this means a post-reshare committee can re-sign any historical `wrap_epoch` identity. Reshare provides *insider rotation* (a previously-compromised share is replaced) but NOT forward secrecy. This is a deliberate design choice — forward secrecy would require re-encrypting every chain-anchored ciphertext on every reshare, which is operationally infeasible. Owner accounts that want forward secrecy can manually rotate by creating a new secret version with `--committee-override` and a fresh committee.

12.4 Collusion of t proxies. `t` colluding proxies can reconstruct `MSK · I` for any identity they choose, decrypting any ciphertext under that release key. CBSS does not prevent this; the on-chain protocol cannot detect off-line collusion. The mitigation is geographic / operator diversity in committee selection (VRF + soak period), staking economics (5 % `InvalidPartialReencrypt` slash on each detected misuse), and CIP-12 governance to ban repeat offenders.

12.5 Owner key compromise. A compromised owner key can submit `UpdateSecretPolicy` to add an attacker actor and pull the secret. CBSS does not prevent this; it is a documented security boundary. Mitigations are wallet-level (hardware keys, multisig) and event-driven (chain emits `cbss.secret.policy_updated` for any change).

12.6 Memory hygiene. Plaintext and DEKs are wrapped in **Zeroizing** at every layer: owner CLI, proxy share storage, runner combiner, runner template substitution, and runner subprocess env. AES-GCM AAD binds (`account`, `key_hash`, `version`, `wrap_epoch`) end to end so any field-level tamper fails authentication.

12.7 Replay protection. Three layers:

- 24-hour proxy-local replay bloom keyed on (`release_id`, `proxy_id`).
- Chain-side `release_receipt:{release_id}` map dedupes by (`release_id`, `proxy_id`) indefinitely.
- `request_block` freshness window `REQUEST_FRESHNESS_BLOCKS` constrains how far back a release request can target.

12.8 Revocation latency. Net bound for “revoked actor’s release fails on chain” = chain finality. Net bound for “no proxy serves a partial under revoked policy” = `finality + PROXY_CACHE_TTL_BLOCKS × block_time`. Honest proxies that already served under stale cache state ate the work; the chain rejects their receipts at submission-time current head.

12.9 TEE composition. When `policy.tee_required = true`, the chain consults the CIP-23 TEE Verifier state at receipt submission and rejects receipts for runners without a valid attestation. Attestation key rotation between secret creation and release is handled correctly because the check is at submission-time current head.

12.10 Side channels. Pairing operations and scalar arithmetic in `cbss-crypto` use `blstrs`’s constant-time primitives. Share scalars use `subtle::CtOption` for branch-free deserialization. HPKE and AES-GCM rely on RustCrypto’s constant-time implementations. External cryptographic review of `cbss-crypto` is a ship gate (§13).

19 13. Parameters

Parameter	Default	Tunable by
<code>MIN_PROXY_STAKE</code>	10,000 token units	Governance
<code>PROXY_SOAK_PERIOD_BLOCKS</code>	86,400 ($\approx$ 30 days)	Governance
<code>PROXY_UNBOND_BLOCKS</code>	86,400 ($\approx$ 30 days)	Governance
<code>DEFAULT_N</code>	5	Governance
<code>DEFAULT_T</code>	3	Governance
<code>MAX_COMMITTEE_N</code>	16	Governance

Parameter	Default	Tunable by
MAX_PRIOR_COMMITTEE_EPOCHS	4	Governance
RESHARE_INTERVAL_BLOCKS	86,400 ($\approx$ 30 days)	Governance
RESHARE_GRACE_BLOCKS	100 ($\approx$ 20 min)	Governance
RESHARE_SAFETY_MARGIN	1	Governance
DKG_BOND	1,000 token units	Governance
DKG_FINALIZE_BLOCKS	7,200 ($\approx$ 1 day)	Governance
DKG_TIMEOUT_SLASH	50 token units	Governance
EXPIRE_DKG_REWARD	1 token unit	Governance
REQUEST_FRESHNESS_BLOCKS	384 ($\approx$ 6 min)	Governance
PROXY_CACHE_TTL_BLOCKS	4 ($\approx$ 4 s)	Local proxy
RELEASE_FEE_PER_RECEIPT	1 token unit	Governance
MAX_RELEASE_OVERDRAFT	$16 \times$ RELEASE_FEE_PER_RECEIPT	Governance
LIVENESS_CHALLENGE_BOND	100 token units	Governance
MIN_CHALLENGE_DELAY_BLOCKS	REQUEST_FRESHNESS_BLOCKS + 32	Governance
LIVENESS_RESPONSE_BLOCKS	64 ($\approx$ 1 min)	Governance
LIVENESS_CHALLENGE_MAX_AGE_BLOCKS	86,400 ($\approx$ 1 day)	Governance
LIVENESS_UNANSWERED_DECREMENT	10BPS	Governance
HEALTH_GOVERNANCE_FLOOR_BPS	1,000	Governance
HEALTH_RECOVERY_BPS_PER_BLOCK		Governance
GOVERNANCE_REVIEW_BLOCKS	2,592,000 ($\approx$ 30 days)	Governance
SLASH_WINDOW_BLOCKS	86,400 ($\approx$ 1 day)	Governance
MAX_SECRETS_PER_ACCOUNT	1,024	Governance
MAX_VERSIONS_PER_SECRET	256	Governance
MAX_ACL_ACTORS	64	Governance
PARTIAL_SIGN_RUNNER_DEADLINE_MS	500	Local runner

20 14. v1 Scope and Phase 2

20.1 14.1 Landed in v1

- Cryptographic primitives (`cbss-crypto`): IBE encrypt/decrypt, threshold partial-sign and combine, PSS reshare, AAD binding, HPKE inter-proxy share envelopes, sealed share storage.
- Chain skeleton (`0x04`): all 14 system instructions, storage shapes, pairing-verified receipt acceptance, prior-committee grace window, signed `RotateCommittee`, signed `runner_sig` and `proxy_sig` recovery on `SubmitReleaseReceipt`, fail-closed `tee_required` and slash evidence (until verifiers are implemented).
- Runner-side `KEY` substitution: `runner-http`, `runner-mcp`, `runner-llm` integration; `SecretBundle` with `Zeroizing` entries; explicit-env subprocess spawn for stdio MCP servers; PVM `os.environ` deny list for declared keys.
- `cbssd` orchestrator state machine for DKG / reshare ceremony progress.

20.2 14.2 Deferred to Phase 2

- **Slash evidence verifiers** for the three fault classes (`InvalidPartialReencrypt`, `DkgSabotage`, `Equivocation`). Today the slash handler fail-closes; auto-slash is non-functional until verifiers land.
- **TEE attestation lookup at receipt submission.** Today `tee_required = true` fail-closes; CIP-23 verifier integration is pending.
- **Bias-free DKG** (`blsful::dkg` migration). The current `cbss-crypto` DKG is rolled-from-scratch Pedersen and biasable; mainnet flag-flip requires the swap.
- **`cbssd` network layer.** The QUIC server, chain watcher, `PartialSign` handler, and receipt submitter are not yet implemented. The orchestrator owns the ceremony state machine but has no transport.
- **Threshold-BLS runner client.** The current `secrets-manager::HttpSecretsClient` is a placeholder that calls a single HTTPS endpoint; the production threshold client (QUIC fanout, Lagrange combine, IBE decrypt) is pending.
- **Liveness deadline transition.** The `ExpireLivenessChallenge` instruction and per-block sweep are pending; today `Pending` challenges do not automatically resolve to `Unanswered`.
- **VRF-based committee selection.** Current selection iterates the proxy index in storage order; full VRF selection (CIP-2 randomness beacon) is pending.
- **Per-instruction gas pricing.** Today `charge_cbss_base` is the only meter; the §8 table is the target.

- **cowboy secrets and cowboy proxy CLI subcommands.** Owner CLI flows are specified; the implementation of the subcommands is pending.
- **OS-keychain operator key sealing.** Today the operator HPKE key is hex on disk at 0o600; macOS Keychain / Linux secret-service integration is pending.
- **Cross-CIP doc updates.** CIP-2, CIP-6, CIP-9, CIP-13 reference patches per CIP-24 §2.3 are pending.

20.3 14.3 Out of Scope (v1)

- Forward-secret reshare (would require re-encryption of all anchored ciphertexts on every rotation).
- Plaintext leak slashing (handled by CIP-12 governance, not on-chain evidence).
- Zero-knowledge proofs of correct partial-sign computation (the pairing check on chain is sufficient for the threat model).
- Hierarchical secrets (sub-keys, scoped namespaces beyond per-account).
- Push-based re-fetch on policy change (runners pull at op time; ACL changes propagate via cache TTL).

21 15. Relationship to the Technical Whitepaper

This document layers on, but never supersedes, the Technical Whitepaper. Specifically:

- Actors, messages, timers, reentrancy, and per-handler resource limits follow Technical Whitepaper §3. The PVM environment-isolation requirement for declared secret keys is an additional sandbox constraint at handler entry.
- Fee markets (Cycles, Cells) and per-block basefee adjustment follow Technical Whitepaper §4 and §17. CBSS instructions consume both meters per §8 of this document.
- State rent for CBSS chain storage (metadata, release keys, receipts, challenges) follows the standard system-actor accounting at 0x04. Off-chain plaintext incurs no rent (it does not exist on chain); CBFS objects holding wrapped payloads bill under CIP-9 storage rent.
- The Runner Marketplace (Technical Whitepaper §5) is the consumer of release requests. Job dispatch integrates with CBSS via the actor's `secrets.read` manifest entitlement; the assigned runner is the only eligible signer of release requests for the job's duration.
- CBFS (CIP-9 / Storage Whitepaper) is the data plane for the encrypted payload. The owner CLI uploads via CapToken at write time; the runner downloads via CapToken at read time. The CBFS pointer on `SecretVersion` is the only chain-resident reference.
- Consensus, randomness, and networking follow Technical Whitepaper §6. Committee selection uses the QC-derived randomness beacon for VRF (Phase 2).

- Governance, upgrades, and system actor hot-code replacement follow Technical Whitepaper §11 and CIP-12. CBSS's mainnet flag-flip is a Tier 0 governance proposal contingent on external cryptographic review of `cbss-crypto`.
- Entitlements (Technical Whitepaper §15) gate `secrets.read` at the manifest level; CBSS is the second, fine-grained layer (per-key, per-version, per-actor).
- TEE attestation (CIP-23) gates release when `policy.tee_required = true`; the chain consults the verifier state at receipt-submission current head.

Where this document and the Technical Whitepaper disagree on values or procedures, the Technical Whitepaper is the normative reference.